

Portfolio Risk & Signal Strategy Tool

PROT — Technical Reference Documentation

Quantitative Analytics Platform for Institutional Portfolio Management

Version 1.0 · June 2026
MATLAB R2023b+ · App Designer

1. Introduction

1.1 Purpose and Scope

The Portfolio Risk & Signal Strategy Tool (PROT) is a professional-grade quantitative analytics platform built in MATLAB for institutional fixed-income and multi-asset portfolio management. It combines two major application workflows — portfolio risk measurement and tactical asset allocation (TAA) from one side, and systematic signal-driven equity-cash rotation strategies from the other — into a single, cohesive codebase backed by a rigorous Model-View-Controller (MVC) architecture.

PROT addresses the complete lifecycle of quantitative portfolio management: sourcing market and macroeconomic data from live APIs, constructing and normalising trading signals, optimising portfolio weights under user-specified constraints, measuring and attributing risk with multiple methodologies, running walk-forward back-tests and conditional bootstrap stress-tests, and presenting results in an interactive, tab-driven App Designer interface.

1.2 Design Philosophy

Three principles govern every design decision in PROT:

- **Causal integrity.** No look-ahead bias is permitted at any level — data normalisation, signal construction, regime labelling, and portfolio optimisation all operate strictly on information available at each historical point in time. Rolling and expanding windows are used consistently; train/test splits enforce temporal ordering.
- **Composability.** The +sigstrat function library exposes fourteen modular, independently testable functions that can be chained arbitrarily. Each function returns a well-specified output (struct, timetable, or numeric matrix) that feeds directly into the next stage without hidden global state.
- **Separation of concerns.** The SignalBuilder sub-framework enforces strict MVC boundaries in code. Model classes contain all business logic and are completely unaware of the UI; View classes are rendering-only; Controller classes orchestrate. This makes the codebase maintainable, testable, and extensible without cross-layer coupling.

1.3 System Requirements

Requirement	Minimum	Recommended
MATLAB	R2021b	R2023b+
Optimization Toolbox	Required	Required
Statistics & Machine Learning Toolbox	Required	Required

Requirement	Minimum	Recommended
Financial Toolbox	Optional	Optional
Datafeed Toolbox (Bloomberg)	Optional — Bloomberg only	Optional
Python	3.9+	3.11+ (for doc generation)
Internet access	Required for live data	Required
FRED API key	Optional (macro data)	Recommended

A free FRED API key can be obtained from fred.stlouisfed.org/docs/api/api_key.html and must be set in the MATLAB environment via `setenv('FRED_API_KEY', 'your_key')` before launching the application. Without a key, macroeconomic data downloads are silently skipped and all macro-derived signals return zero or NaN.

2. System Architecture

2.1 High-Level Component Map

PROT is organised into three top-level namespaces plus two standalone applications. The diagram below shows the principal components and their dependencies.

Layer	Namespace / File	Role	Key Classes / Functions
Application	PortfolioRiskOptimizerApp.m	Risk & allocation UI (9 tabs)	App Designer uifigure
Application	SignalStrategyApp.m	Signal strategy UI (6 tabs)	App Designer uifigure
Library	+sigstrat	Core analytics pipeline (14 functions)	downloadMarketData, buildSignal, optimizeComposite, ...
Framework — Model	SignalBuilder/+Model	Data access & signal engine (9 classes)	DataManager, MarketData, SignalEngine, Backtester, ...
Framework — Controller	SignalBuilder/+Controller	Orchestration layer (4 classes)	DataController, StrategyController, ...
Framework — View	SignalBuilder/+View	Rendering layer (4 classes)	ChartView, StrategyView, ...
Testing	SignalBuilder/+Tests	Unit test suite (5 classes, 153 tests)	TestMarketData, TestSignalEngine, TestBacktester, ...
Utilities	GetVolSurface.m	SPY options vol surface calibration	SVI calibration, IV interpolation
Specialised	SignalBuilder/Underweight/	Equity underweight regime detection	12 scripts, decision tree & threshold logic

2.2 Model-View-Controller Pattern

The SignalBuilder sub-framework enforces a strict three-layer MVC architecture, governed by the rules documented in SignalBuilder/CLAUDE.md. Compliance with these rules is non-optional and is verified by the unit test suite.

The three layers interact as follows:

- **Model (+Model).** All business logic lives here. Model classes inherit from the MATLAB handle superclass, enabling shared-reference semantics across controllers and views. The canonical time-series container is the MATLAB timetable type (never parallel arrays). Models are completely unaware of any UI element and may only communicate outward through

MATLAB events (`notify/addlistener`).

- **View (+View).** Views are rendering-only. They consume Model data through properties and produce MATLAB figures or uifigure panels. They never mutate Model state directly.
- **Controller (+Controller).** Controllers instantiate Models and Views, wire event callbacks, orchestrate the data-flow pipeline, and manage application lifecycle. They are the only layer permitted to call methods on both Models and Views simultaneously.

Architectural invariant

A +Model class that imports or references any +View or App Designer class is a design error. This boundary is intentional: it allows the entire analytics core to be tested headlessly, in batch, or via command-line without launching any UI.

2.3 Naming Conventions

Category	Convention	Example
Package class	PascalCase	MarketData, StrategyView
Method / callback	camelCase	loadData(), computeSignal()
Public property	camelCase	priceData, signalTable
Constant	UPPER_SNAKE_CASE	MAX_LOOKBACK
Script / entry point	lower_snake or camelCase	run_PortfolioRiskOptimizerApp.m
Error identifier	Package:Class:ID	SignalBuilder:DataManager:NotFound

3. Data Infrastructure

Reliable, bias-free data acquisition is the foundation of any quantitative system. PROT provides a multi-source data layer with intelligent caching, automatic gap-filling, and MATLAB timetable standardisation at every boundary.

3.1 DataManager Class

`Model.DataManager` is the central hub for all external data access. It abstracts five distinct API sources behind a uniform interface and manages a two-tier persistence layer:

Tier 1 — In-memory registry: FRED macro series cached for 24 hours with last-fetch timestamp. Prevents redundant HTTP calls within a session.

Tier 2 — Persistent .mat files: OHLCV timetables cached in `cache/mat/* .mat`. A companion SQLite database (`cache/metadata.db`) tracks ticker, last-date, source, and file path for each cached series.

Source	Method	Data Type	Auth	Rate Limit
Yahoo Finance	<code>fetch_yahoo()</code>	Daily OHLCV (equities, ETFs, indices)	None	~2 req/s (429 auto-retry)
Stooq	<code>fetch_stooq()</code>	Daily OHLCV (equities, T-bill proxy)	None	Generous
FRED	<code>fetch_fred()</code>	Monthly macro series (GDP, CPI, VIX, ...)	Free API key	120 req/min
Alpha Vantage	<code>fetch_alphavantage()</code>	Daily adjusted OHLCV	Free API key	5 req/min
Bloomberg	<code>fetch_bloomberg()</code>	Any Bloomberg field via BLP terminal	Active session	N/A

The standardised output of every fetch method is a MATLAB timetable with:

- Row times of type `datetime` with time zone `America/New_York` for equity series.
- Variable names in PascalCase: `Open`, `High`, `Low`, `Close`, `Volume`.
- NaN sentinel for any missing observation; gap-filling is applied post-fetch using one of three configurable methods: `'linear'`, `'previous'`, or `'none'`.
- Multi-asset synchronisation via MATLAB's `synchronize(..., 'union', 'fillwithconstant', NaN)`.

3.2 Bloomberg Integration

Bloomberg data is accessed through the MATLAB Datafeed Toolbox `blp()` connection object and the `history()` function. The implementation handles two critical requirements of the modern

BLPAPI interface:

- **Date format:** Bloomberg expects dates as 'mm/dd/yyyy' character strings, not MATLAB datenums. The app converts internal yyyyMMdd strings via `datestr(datetime(d, 'InputFormat', 'yyyyMMdd'), 'mm/dd/yyyy')`.
- **Overrides format:** Newer BLPAPI versions require overrides as an N×2 cell array, not trailing name-value positional arguments. PROT passes a structured cell array specifying `periodicitySelection`, `periodicityAdjustment`, `nonTradingDayFillOption`, and `nonTradingDayFillMethod`.

```
bbgOverrides = {'periodicitySelection',    bbgFreq;
               'periodicityAdjustment',   'CALENDAR';
               'nonTradingDayFillOption', 'ALL_CALENDAR_DAYS';
               'nonTradingDayFillMethod', 'PREVIOUS_VALUE'};
d = history(c, ticker, field, d1, d2, bbgOverrides);
```

3.3 FRED Macroeconomic Data Pipeline

`sigstrat.downloadMacroData` retrieves twelve primary FRED series and derives fourteen additional engineered features from them, covering the major macroeconomic regimes relevant for equity-cash rotation strategies.

FRED Series ID	Description	Derived Features
T10Y2Y	10Y-2Y Treasury yield spread	YieldSlope_Z (z-score)
BAMLH0A0HYM2	HY OAS credit spread (%)	CreditSpread_Z (winsorised z-score)
PAYEMS	Total nonfarm payrolls (thousands)	Payrolls_MoM_Z (momentum z-score)
VIXCLS	CBOE VIX spot	VIX_Z (rolling z-score)
DTWEXBGS	USD broad trade-weighted index	DXY_Z
DFF	Federal funds rate	FFR, FFR_Change
GS10	10-year Treasury yield	Yield10Y, YieldChange
CPIAUCSL	CPI (all urban, SA)	CPI_YoY
GDPC1	Real GDP (quarterly)	GDP_Growth_YoY
UMCSENT	University of Michigan sentiment	Sentiment_Z
RSXFS	Retail sales ex-food service	RetailSales_YoY
DCOILWTICO	WTI crude oil price	Oil_Z, Oil_YoY

All derived features are computed using causal (backward-looking) rolling windows. Year-over-year percentage changes use a fixed 252-trading-day lag. Z-score normalisation uses an expanding or rolling window of configurable length (default 60 months). This ensures that no information from future observations contaminates any signal value.

3.4 Macro Signal Library

Three production-grade economic indicator functions are implemented in +Model, each following an identical four-stage pipeline:

Stage 1 — Raw fetch. FRED series is downloaded via DataManager, aligned to the requested business-day grid, and forward-filled for weekends and holidays.

Stage 2 — Winsorisation. An expanding-window percentile clip (default 97th percentile) prevents extreme episodes (GFC 2008, COVID-19 2020) from permanently compressing the signal into a narrow band. The clip boundary updates at each time step using only past data.

Stage 3 — Rolling z-score. A causal rolling window (default 60 months) computes mean and standard deviation. The z-score is computed with a minHistory guard to avoid division by zero in the warm-up period.

Stage 4 — EMA smoothing and tanh compression. An exponential moving average (configurable half-life, default 3 months) dampens high-frequency noise. Final output is compressed to $[-1, +1]$ via the hyperbolic tangent, making the signal gradient-friendly for downstream optimisers.

Signal Function	Input Series	Economic Interpretation	Key Params
yieldCurveSlope	T10Y2Y	Positive = steep curve (bullish growth regime); Negative = flat/inverted (recession signal)	ZScoreWindow=60mo, SmoothHL=3mo
creditSpreadSignal	BAMLH0A0HYM2	Inverted: wide spread (stress) → negative; tight spread (benign) → positive	ZScoreWindow=60mo, WinsorPct=97
payrollsMomentum	PAYEMS	Positive = hiring boom (risk-on); Negative = job losses (risk-off)	PrefilterWindow=3mo, NormWindow=60mo

4. Portfolio Risk Optimizer Application

PortfolioRiskOptimizerApp.m is the primary institutional portfolio management interface. It provides nine integrated analysis tabs covering risk measurement, portfolio optimisation, strategic and tactical allocation, tracking error attribution, conditional stress testing, and performance analytics.

4.1 Application Layout

The application uses an App Designer uifigure split into a fixed left control panel and a right-side tab group.

Tab	Primary Components	Function
Summary	SummaryText, WeightsTable	Portfolio-level metrics: VaR, CVaR, ETL, annualised return & vol, Sharpe
Distribution	DistAxes (uiaxes)	Return distribution histogram with KDE overlay and VaR/CVaR markers
Sensitivity	SensAxes	+1% per-asset shock impact chart; linear approximation using asset returns
Scenarios	ScenTable	Named scenario PnL approximations (GFC, COVID, Rate Shock, custom)
Data	DataTable	Raw input price/return matrix viewer with date and ticker dimensions
Allocation (SAA)	SAATable, SAAMetricsText, SAASStressTable	Strategic Asset Allocation: weights, portfolio metrics, historical stress results
TAA	TAAAssetsList, TAATable, TAAAddTradeBtn	Tactical overlays: asset-level delta positions, trade pair builder
Combined	CombinedTable, TEMethodDropDown, TETable	SAA+TAA merged weights; tracking error vs benchmark with TE attribution
Bootstrap	BootstrapSummary, BootstrapAxes, BootCondTable	Conditional historical block bootstrap: define regime conditions, run & visualise
Performance	PerfTable, PerfAxesCum, PerfAxesVol	Rolling performance analytics: Sharpe, vol, return over configurable window

4.2 Risk Measurement Methods

Value at Risk (VaR) and related tail measures are computed using three methods:

Historical Simulation. The empirical quantile of the return distribution at confidence level α is taken directly from the sorted historical return vector. No distributional assumption is made. Effective for capturing fat tails and skewness present in actual asset returns.

Parametric (Variance-Covariance). Portfolio return is assumed to follow a normal distribution $N(\mu_p, \sigma_p^2)$. $\text{VaR}_\alpha = -\mu_p + z_\alpha \cdot \sigma_p$, where z_α is the standard normal quantile. The covariance matrix Σ is estimated from the full available history using the sample covariance estimator.

Monte Carlo Simulation. 10,000 portfolio return paths are drawn from a multivariate normal distribution parameterised by the sample mean vector and covariance matrix. VaR and CVaR are computed as empirical quantiles of the simulated portfolio return distribution.

Conditional VaR (CVaR, also known as Expected Shortfall or ETL) is always computed as the mean of returns falling below the VaR threshold, regardless of the VaR method selected. CVaR is a coherent risk measure satisfying sub-additivity and convexity properties not guaranteed by VaR alone.

4.3 Portfolio Optimisation

The optimiser minimises or maximises an objective function subject to linear equality (full-investment) and inequality (weight bounds) constraints. Five objective functions are implemented:

Objective	Mathematical Formulation	Solver
MinVar	$\min w' \Sigma w \text{ s.t. } 1'w=1, l \leq w \leq u$	quadprog
MaxSharpe	$\max (\mu'w - r_f) / \sqrt{w' \Sigma w} \text{ s.t. } \dots$	fmincon (interior-point)
MinCVaR	$\min \text{CVaR}_\alpha(w) \text{ via Rockafellar-Uryasev LP reformulation}$	linprog
MinVaR	$\min \text{VaR}_\alpha(w) \text{ (historical simulation)}$	fmincon
ERC (Risk Parity)	$\min \sum_{i,j} (RC_i - RC_j)^2 \text{ where } RC_i = w_i (\Sigma w)_i$	fmincon

5. Strategic & Tactical Asset Allocation Framework

The SAA/TAA framework implements a two-layer portfolio construction process that separates long-term strategic allocation from short-term tactical overlays, while providing full risk attribution between the two layers.

5.1 SAA Layer

The Strategic Asset Allocation (SAA) represents the long-run target weights, typically derived from a liability-matching framework, a mean-variance efficient frontier, or an investment policy statement. In PROT, SAA weights are loaded from the same data file as the portfolio returns. The SAATable allows the user to manually override weights subject to the normalisation constraint $\sum w_i = 1$.

The SAA Metrics panel computes the full risk and return profile of the SAA portfolio: expected return, annualised volatility, Sharpe ratio, VaR (at the configured method and confidence level), and a stress test panel applying the six canonical historical shocks defined in Section 7.4.

5.2 TAA Layer

Tactical overlays are expressed as delta positions on top of the SAA weights. Each row in the TAATable represents a single position leg with the following fields:

Column	Type	Description
TradeGroup	string	Identifier grouping related legs (e.g., "Trade1")
Asset	string	Ticker or asset name matching the SAA universe
DeltaPct	double	Weight delta in decimal (e.g., 0.05 = +5%)
Leg	string	Auto-derived: "Long" if DeltaPct>0, "Short" if <0
Description	string	Optional free-text trade description
Standalone_TE	double	Tracking error of this position in isolation (bps)
Marginal_TE	double	Incremental TE contribution to total portfolio TE (bps)
DivBenefit	double	Standalone_TE – Marginal_TE (bps); measures diversification
EqBeta	double	Equity beta sensitivity of the position
USD_Sens	double	USD sensitivity (for multi-currency portfolios)

5.3 Tracking Error Methodology

Tracking error (TE) measures the annualised standard deviation of active returns (portfolio minus benchmark). PROT implements three methods, selectable via the TEMethodDropDown control in the Combined tab:

Variance-Covariance (default).

Given the active weight vector $w_{\text{diff}} = w_{\text{TAA}} - w_{\text{SAA}}$ and the covariance matrix Σ estimated from historical returns:

```
TE = sqrt(w_diff' * Sigma * w_diff) * sqrt(252)
```

This is the standard ex-ante TE estimate. It is fast and analytically tractable but assumes a stationary covariance structure.

Historical.

Realised active returns are computed as $r_p - r_{\text{bm}}$ on each observation date. TE is the annualised standard deviation of this series:

```
active_ret = R * w_TAA - R * w_SAA;
TE = std(active_ret) * sqrt(252);
```

This method is backward-looking and reflects actual portfolio behaviour including any non-normality or time-variation in correlations.

Exponentially Weighted (EWMA, $\lambda=0.94$).

The covariance matrix is estimated using an exponentially weighted scheme with decay factor $\lambda = 0.94$ (the RiskMetrics standard). More recent observations receive higher weight, making the estimate more responsive to recent volatility regimes:

```
function S = ewcov(R, lambda)
    [T, N] = size(R);
    w = lambda .^ (T-1:-1:0)';           % Tx1 weight vector
    w = w / sum(w);                     % normalise to sum to 1
    mu = w' * R;                        % weighted mean
    Rc = R - repmat(mu, T, 1);          % demeaned returns
    S = Rc' * diag(w) * Rc;              % weighted outer product
    S = (S + S') / 2;                   % enforce symmetry
end
```

5.4 Risk Attribution (Euler Decomposition)

For each asset i and for each trade group, PROT computes three risk contribution metrics using the Euler decomposition of portfolio variance. All three are expressed in basis points.

Standalone Tracking Error. The TE of the position considered in isolation — i.e., as if it were the only active position in the portfolio:

```
w_isolated = zeros(N, 1);
w_isolated(i) = w_diff(i);             % single active position
TE_standalone = sqrt(w_isolated' * Sigma * w_isolated) * sqrt(252);
```

Marginal Tracking Error. The incremental TE contribution — i.e., the reduction in total portfolio TE that would result from removing position i :

```

w_without = w_diff;
w_without(i) = 0; % remove position i
TE_without = sqrt(w_without' * Sigma * w_without) * sqrt(252);
TE_marginal = TE_all - TE_without;

```

Diversification Benefit. The difference between standalone and marginal TE, quantifying how much the position's risk is offset by its correlation with other active positions:

```
DivBenefit = TE_standalone - TE_marginal;
```

A large diversification benefit ($\text{DivBenefit} \gg 0$) indicates that the position hedges other active bets. A negative diversification benefit indicates that the position adds more TE than it would in isolation — it is correlated with (rather than offsetting) existing risk exposures.

6. Signal Strategy Application

SignalStrategyApp.m provides a complete workflow for systematic equity-cash rotation strategies driven by macro and momentum signals. The application supports three strategy modes and integrates the full +sigstrat analytics pipeline.

6.1 Signal Construction Pipeline

All signals are scalar time series bounded in $[-1, +1]$, where $+1$ represents maximum risk-on conviction (100% equity) and -1 represents maximum risk-off conviction (0% equity). The pipeline has four stages:

Stage 1 — Raw Signal Generation.

Six canonical signal types are supported via `sigstrat.buildSignal`:

Type	Description	Key Parameters
Momentum	Return over rolling lookback window, vol-normalised	lookback, volHalflife, skipDays
EWMAC	Diff of fast and slow EMA (exponential crossover)	fastSpan, slowSpan, volFloor
Ensemble	Equal-weight or custom-weight average of multiple signals	weights, method
MacroZ	Rolling z-score of any macro series from <code>downloadMacroData</code>	series, zWindow, activation
MeanReversion	Deviation from long-run mean, mean-reverting bet	lookback, entryZ, exitZ
Composite	Weighted composite of multiple sub-signals	subSignals, weights

Stage 2 — Normalisation. Five normalisation methods are available in `sigstrat.normalizeSignals`:

Method	Formula	Best Used When
RobustZ	$(x - \text{median}) / \text{MAD}$, then $\tanh$	Outlier-prone raw signals
StandardZ	$(x - \mu) / \sigma$, then $\tanh$	Approximately normal signals
RollingZ	Rolling $(x - \mu_w) / \sigma_w$, then $\tanh$	Non-stationary signals
Percentile	Empirical CDF rank $\rightarrow [0,1] \rightarrow$ scale to $[-1,+1]$	Heavy-tailed signals
MinMax	$2 \cdot (x - \min) / (\max - \min) - 1$	Bounded signals only

Stage 3 — Signal-to-Weight Mapping. The normalised signal in $[-1, +1]$ is mapped to an equity weight in $[0, 1]$ via `sigstrat.mapSignalToWeight`:

Method	Description
Step	Piecewise constant: thresholds partition $[-1, +1]$ into discrete weight levels
Linear	Simple linear interpolation: $w = 0.5 \cdot (1 + \text{signal})$
Sigmoid	$w = 1 / (1 + \exp(-k \cdot \text{signal}))$; k controls steepness
PiecewiseLinear	User-supplied breakpoints (x, w) with linear interpolation
Spline	Cubic spline through user-supplied control points
Power	$w = \text{sign}(s) \cdot s ^p$ rescaled to $[0, 1]$; p controls convexity

6.2 Strategy Modes

Three optimisation modes are available, each suited to different levels of modelling complexity and explainability requirements.

Mode 1 — Composite Signal

A weighted combination of normalised signals is passed through a step function with K thresholds to produce a discretised equity weight. `sigstrat.optimizeComposite` jointly optimises the signal weights and threshold values to maximise the selected performance objective (Sharpe, Sortino, or Calmar) subject to a 70/30 in-sample/out-of-sample temporal split. The optimiser uses `fmincon` with multiple random initialisations to avoid local optima.

Mode 2 — Risk Budget

`sigstrat.riskBudgetStrategy` allocates a fraction of total active risk budget to each signal. Each signal is independently translated from its normalised value to an equity weight contribution using any of the seven mapping methods. The final equity weight is the budget-weighted sum of per-signal contributions. An optional macro conditioning layer allows the risk budget to tighten (or widen) based on a specified macro indicator breaching a threshold — for example, reducing equity allocation when the yield curve is inverted.

Mode 3 — Decision Tree

`sigstrat.decisionTreeStrategy` supports two sub-modes:

Rule-Based. The user specifies an $R \times 4$ rule matrix $\{\text{varName}, \text{operator}, \text{threshold}, \text{eqWeight}\}$. Rules are evaluated sequentially; the first matching rule determines the equity weight for that period. This approach maximises interpretability and is suitable for translating discretionary views into systematic rules.

Auto-Optimise. MATLAB's `fitctree` classifier is trained on lagged signal features to predict equity weight quantile labels. Hyperparameters (`maxDepth`, `minLeafSize`) are swept over a

configurable grid. Expanding-window cross-validation (3 folds) guards against overfitting. Variable importance scores are reported to support signal selection.

6.3 Backtesting Engine

`sigstrat.backtestEquityCash` implements a realistic walk-forward backtest of a two-asset (equity + cash) strategy:

- Equity allocation at time t is determined by the strategy signal at $t-1$ (one-step-ahead rule to prevent look-ahead bias).
- Rebalancing occurs every *rebalFreq* business days (default: 21, approximately monthly).
- Transaction costs of *txCost* basis points per unit of turnover are applied at each rebalance.
- Position constraints: *eqMin*, *eqMax*, and optional maximum drawdown limit are enforced.
- Benchmark portfolios (60/40 equity-cash and 100% equity) are computed alongside the strategy for comparison.
- Output struct includes daily portfolio returns, cumulative equity curve, equity weight series, turnover, and dates.

6.4 Performance Measurement

`sigstrat.computeStrategyPerf` computes a comprehensive performance profile for the strategy and both benchmarks. The following metrics are reported:

Metric	Definition
Ann. Return (%)	$(\Pi(1+r_t))^{252/T} - 1$, annualised
Ann. Volatility (%)	$\sigma(r_t) \cdot \sqrt{252}$
Sharpe Ratio	$(\text{Ann. Return} - r_f) / \text{Ann. Vol}$
Sortino Ratio	$(\text{Ann. Return} - r_f) / \text{Downside Vol}$
Calmar Ratio	$\text{Ann. Return} / \text{Max Drawdown}$
Max Drawdown (%)	$\max_t (\text{CumMax}_t - \text{CW}_t) / \text{CumMax}_t$
Hit Rate (%)	Proportion of daily returns > 0
Skewness	Third standardised moment of daily returns
Excess Kurtosis	Fourth standardised moment minus 3

7. Bootstrap Stress Testing

7.1 Methodology

The bootstrap stress testing module in the PortfolioRiskOptimizerApp implements a **conditional block bootstrap** method. Unlike a simple Monte Carlo simulation that draws independent return shocks from a fitted distribution, the block bootstrap preserves the autocorrelation and cross-sectional dependency structure present in actual financial return data — properties that are especially important during stress episodes.

A *block* is a contiguous segment of L consecutive trading days drawn at random from the historical return matrix. Concatenating B randomly selected blocks produces one simulated path of approximately $B \cdot L$ days. The portfolio NAV is computed along each simulated path, yielding a distribution of future cumulative returns from which VaR, CVaR, and maximum drawdown percentiles are extracted.

The *conditional* prefix means that the user may restrict the bootstrap sampling universe to only those historical blocks that satisfy user-specified regime conditions (e.g., "yield curve inverted AND credit spreads above 500 bps"). This allows the user to ask counterfactual questions such as: "What would be my portfolio VaR if the macro environment resembled the 2007–2009 GFC period?"

7.2 Regime Condition Builder

Conditions are defined as logical expressions over the measures loaded by the LoadMeasuresButton. Each condition specifies a variable, a comparison operator ($>$, $<$, $>=$, $<=$, $==$, $!=$), and a threshold value. Multiple conditions are combined using a global AND or OR logic (configurable via BootCondLogicDrop). Only historical dates matching all (AND) or any (OR) condition are eligible for block sampling.

Example condition Variable: VIX_Z · Operator: $>$ · Threshold: 1.5 → "Select only high-stress periods when VIX was more than 1.5 standard deviations above its rolling mean."
Combined with: CreditSpread_Z $>$ 1.0 (AND), this restricts sampling to simultaneous equity fear and credit stress episodes.

7.3 Configuration Parameters

Parameter	Default	Description
Block Length (L)	21 days	Length of each bootstrap block. Longer blocks preserve more autocorrelation; shorter blocks provide more path diversity.
Paths (B)	1,000	Number of simulated paths. Higher values improve distributional accuracy at the cost of compute time.

Parameter	Default	Description
Horizon	From VaR tab	Forecast horizon in days; determines number of blocks per path as $\text{ceil}(\text{Horizon} / L)$.
Condition Logic	AND	AND: all conditions must hold. OR: any condition must hold.

7.4 Historical Stress Episodes

`sigstrat.stressTestStrategy` evaluates strategy performance across six canonical historical market stress events:

Episode	Approximate Dates	Characteristic Shock
Global Financial Crisis	2008-09 to 2009-03	Systemic deleveraging; equity –55%, HY spreads +2000bp
European Debt Crisis	2011-07 to 2011-09	Sovereign spread widening; peripheral contagion
Taper Tantrum	2013-05 to 2013-06	Sudden yield spike on Fed tapering signal
Q4 2018 Selloff	2018-10 to 2018-12	Equity drawdown on Fed overtightening fears
COVID-19 Crash	2020-02 to 2020-03	Fastest 30% equity drawdown on record
Inflation / Rate Shock	2022-01 to 2022-06	Simultaneous equity and bond losses; 60/40 worst year since 1970

8. SignalBuilder MVC Framework

The SignalBuilder sub-framework is a self-contained MVC implementation that provides the foundational infrastructure for the SignalStrategyApp and for programmatic use of the PROT analytics stack without launching any UI. It lives entirely within the `SignalBuilder/` directory and is independently deployable.

8.1 Model Layer

Class	Inherits	Responsibility
BaseModel	handle	Abstract base: Name property, structured logging (logInfo/logError), event system, timetable validation helpers
MarketData	BaseModel	OHLCV storage in a TZ-aware timetable. Properties: Ticker, DateRange, NumBars. Methods: loadFromFile, loadFromTimetable, getPrices, getSimpleReturns, getLogReturns
DataManager	BaseModel	Multi-source data acquisition + SQLite-backed caching. init() opens/creates metadata.db. Supports FRED, Yahoo, Stooq, Alpha Vantage, Bloomberg. Configurable: Timeout, FillMethod, Verbose
Signal	BaseModel	Technical indicator builder wrapping a MarketData handle. Methods: addSMA, addEMA, addRSI, addCrossoverSignal. Auto-refreshes on upstream MarketData change via listener
SignalEngine	BaseModel	Vectorised transform library + multi-signal aggregation. Public Params struct allows optimisers to mutate hyperparameters. Aggregation: static_weights, risk_parity (inverse vol), logic_gate (conditional activation)
Backtester	BaseModel	Signal-driven backtest engine + Bayesian hyperparameter optimisation. Constraints: MaxLeverage, MaxWeightPerAsset. Metrics: Sharpe, Sortino, MaxDD, Turnover, TotalReturn
Strategy	BaseModel	Entry/exit rule set + backtest runner. Methods: setEntryRule, setExitRule, run, getMetrics. Produces BacktestResults timetable
Portfolio	BaseModel	Portfolio construction and analytics stub (extensible)

8.2 Controller Layer

Class	Responsibility
BaseController	Abstract base: Name property, managed listener array (addManagedListener/shutdown), auto-cleanup destructor. Abstract methods: init(), run()
DataController	Owns DataManager + MarketData models. Manages file loading via containers.Map ticker registry. Fires DataLoaded event on update

Class	Responsibility
StrategyController	Wires Signal + SignalEngine + Backtester into signal-strategy pipeline. Fires StrategyComplete on success
UnderweightController	Equity underweight regime detection and two-tranche execution strategy. Methods: computeRegimes, executeTrades, loadMacroData

8.3 View Layer

Class	Responsibility
BaseView	Abstract base: Figure property, Title, show/hide/close lifecycle, IsRendered flag. Abstract: createFigure
ChartView	OHLCV price data + signal overlay charts from timetables. Methods: setPriceData, setSignalData, setEquityData, render
StrategyView	2-panel performance summary (metrics table + drawdown chart). Methods: setResults(resultsTT, metrics, strategyName), render
UnderweightApp	uifigure-based UI for underweight regime analysis and trade execution. Tabs: regime analysis, signal display, trade execution

8.4 Bayesian Hyperparameter Optimisation

The `Backtester.optimize` method wraps MATLAB's `bayesopt` for automated hyperparameter tuning of the signal pipeline. The user specifies which `SignalEngine.Params` fields are optimisable and their search bounds. The optimiser maximises the Sharpe ratio computed on a held-out validation window using a Gaussian process surrogate model.

The acquisition function defaults to 'expected-improvement-plus', which balances exploration (sampling uncertain regions) with exploitation (sampling near the current best). The number of evaluations, maximum time, and verbose logging level are configurable via `OptimOptions`.

9. Underweight Strategy Module

The Underweight module (`SignalBuilder/Underweight/`) implements a specialised workflow for detecting equity underweight regimes — periods during which the multi-signal framework recommends below-neutral equity exposure — and translating those signals into concrete portfolio actions.

9.1 Signal Construction

`buildSignals.m` loads four fundamental signals via `DataManager` and aligns them to a monthly timetable through `resampleMonthly.m`:

Signal	Source	Interpretation
Payrolls Momentum	FRED PAYEMS	Labour market health; leading indicator for equity risk appetite
Yield Curve Slope	FRED T10Y2Y	Business cycle phase; inversion historically precedes recessions
Credit Spread	FRED BAMLH0A0HYM2	Financial conditions; widening spreads signal tightening liquidity
Equity Trend	Yahoo SPY price	Price momentum; sub-200-day MA signals deteriorating trend

9.2 Decision Logic

Two complementary approaches are available for deriving the underweight decision from the four signals:

Threshold Approach (`thresholdApproach.m`): Each signal is thresholded independently. An underweight is triggered when a configurable number of signals breach their respective thresholds simultaneously (e.g., "go underweight when at least 3 of 4 signals are negative").

Decision Tree Approach (`decisionTreeApproach.m`): A CART classifier is trained to predict binary underweight/neutral labels from the signal features. The tree depth and minimum leaf size are optimised to balance in-sample fit against generalisation. The resulting rules are extracted as human-readable text for auditability.

9.3 Regime Visualisation

`shadeRegimes.m` renders underweight periods as shaded vertical bands on an equity price chart, making it easy to visually assess the timing and duration of risk-off signals relative to major market events. `labelEquityRegimes.m` classifies each date into one of four equity market states (Bull Trend, Bear Trend, Recovery, Consolidation) based on price momentum and volatility quantiles.

10. Volatility Surface Utility

`GetVolSurface.m` provides an SPY options-based implied volatility surface calibrated using the Stochastic Volatility Inspired (SVI) parametric model. This enables options pricing, hedging, and risk scenario analysis within the same framework.

10.1 Data Source

SPY options chains are downloaded from Yahoo Finance's options API, which provides real-time bid/ask quotes, open interest, and implied volatilities for all listed expiries. A 24-hour cache prevents redundant downloads. The function accepts a target (T, K) pair and returns the interpolated implied volatility.

10.2 SVI Calibration

The SVI (Stochastic Volatility Inspired) model parametrises the implied variance smile as a function of log-moneyness $k = \ln(K/F)$ for each expiry slice T:

$$w(k) = a + b \cdot (\rho \cdot (k - m) + \sqrt{(k - m)^2 + \sigma^2})$$

where $w(k)$ is total implied variance, and $\{a, b, \rho, m, \sigma\}$ are the five SVI parameters calibrated by minimising mean squared error against market mid-quotes subject to the no-butterfly-arbitrage constraints ($b \geq 0$, $|\rho| < 1$, $\sigma > 0$).

The function supports both absolute strike K and log-moneyness k input. For interpolation between expiries, a cubic spline is fitted to the SVI parameters across the term structure.

11. Testing & Validation

PROT ships with two complementary test suites: a regression integration suite covering end-to-end data sourcing and application startup, and a modular unit test suite covering all SignalBuilder Model classes.

11.1 Integration Test Suite (run_prot_tests.m)

This script runs 18 integration checks organised into 10 sections. Tests are run with explicit rate-limit pauses to respect Yahoo Finance and FRED API constraints.

Section	Checks	Pass Criteria
1. Yahoo Finance direct	3 (SPY, ^GSPC, BIL)	> 400 rows per ticker
2. sigstrat.downloadMarketData	1	> 400 trading days, SPX annual return plausible
3. DataManager.fetch_stooq	2 (SPY, ^SPX)	> 200 rows; 429 rate-limit treated as WARN not FAIL
4. DataManager.fetch_yahoo	1 (QQQ)	> 200 rows
5. FRED REST API reachability	1	400 Bad Request = network OK (invalid key rejected correctly)
6. sigstrat.downloadMacroData	1	>= 10 columns returned
7. sigstrat analytics pipeline	6	generateDemoSignals → normalizeSignals → mapSignalToWeight → backtestEquityCash → computeStrategyPerf; each step passes
8. SignalBuilder unit tests	5 test files	0 test failures across all 153 tests
9. App instantiation	2	PortfolioRiskOptimizerApp and SignalStrategyApp instantiate without error; UIFigure is valid

Target: 18 PASSED / 0 FAILED. Run from the project root:

```
cd C:\CIO\ClaudeCode\RiskToolpublic
run_prot_tests
```

11.2 Unit Test Suite (+Tests)

Test Class	Coverage Areas	Test Count (approx.)
TestMarketData	Construction, timetable loading, timezone enforcement, date range filtering, simple and log return computation, error paths	~30
TestSignal	SMA, EMA, RSI, crossover signal accuracy against known reference values	~20

Test Class	Coverage Areas	Test Count (approx.)
TestSignalEngine	Transform output accuracy (MA, z-score, RS, VNR), Params struct mutability, aggregation methods (static, risk_parity, logic_gate), error paths	~40
TestBacktester	Equity curve calculation, all 5 metrics, constraint enforcement (max leverage, min/max weight)	~35
TestStrategy	Entry/exit rule evaluation, full backtest execution with entry/exit logic, metrics computation	~28

Run the unit tests from the SignalBuilder directory:

```
results = runtests('+Tests');
table(results)
```

11.3 Key Bug Fixes Incorporated

Issue	Root Cause	Fix Applied
mapSignalToWeight "Too many input arguments"	Called as (signal, 0.5) — numeric treated as method string	Changed to (signal, 'Sigmoid')
all() "Too many input arguments"	MATLAB all() does not accept 'omitnan' option	Replaced with: wFin=w(isfinite(w)); all(wFin>=0) && all(wFin<=1)
"Unrecognized variable bt"	bt never initialised when backtestEquityCash block was skipped	Added bt = []; before the conditional block
Bloomberg "STRING input" error	datetime() returns double; Bloomberg expects mm/dd/yyyy char string	Convert via datestr(datetime(...), 'mm/dd/yyyy')
Bloomberg overrides format error	Newer BLPAPI rejects trailing name-value pairs; requires N×2 cell	Restructured overrides as N×2 cell array
TAA table vertical concatenation error	New rows built with 5 columns; existing table had 10 after columns were added	Dynamic padding: setdiff(Tcur.VariableNames, newRow.VariableNames)

12. Configuration & Deployment

12.1 First-Time Setup

```
% 1. Set FRED API key (get free key at fred.stlouisfed.org/docs/api/api_key.html)
setenv('FRED_API_KEY', 'your_key_here')

% 2. Ensure required toolboxes are installed
ver('Optimization Toolbox')
ver('Statistics and Machine Learning Toolbox')

% 3. Launch the Portfolio Risk application
run_PortfolioRiskOptimizerApp

% 4. Launch the Signal Strategy application
run_SignalStrategyApp

% 5. Run the full test suite to verify your installation
run_prot_tests
```

12.2 Project Directory Structure

```
RiskToolpublic/
■■■■ PortfolioRiskOptimizerApp.m    % Primary risk / allocation application
■■■■ SignalStrategyApp.m           % Signal strategy application
■■■■ run_PortfolioRiskOptimizerApp.m
■■■■ run_SignalStrategyApp.m
■■■■ run_prot_tests.m              % Integration test runner
■■■■ GetVolSurface.m              % SVI volatility surface utility
■■■■ +sigstrat/                   % Core analytics package (14 functions)
■   ■■■■ downloadMarketData.m
■   ■■■■ downloadMacroData.m
■   ■■■■ buildSignal.m
■   ■■■■ generateDemoSignals.m
■   ■■■■ generateExtendedSignals.m
■   ■■■■ normalizeSignals.m
■   ■■■■ mapSignalToWeight.m
■   ■■■■ backtestEquityCash.m
■   ■■■■ computeStrategyPerf.m
■   ■■■■ stressTestStrategy.m
■   ■■■■ optimizeComposite.m
■   ■■■■ riskBudgetStrategy.m
■   ■■■■ decisionTreeStrategy.m
■   ■■■■ robustOptimize.m
■■■■ SignalBuilder/              % MVC framework
■   ■■■■ MainApp.m               % 4-tab standalone app
■   ■■■■ CLAUDE.md               % Architectural rules
■   ■■■■ +Model/
■   ■■■■ BaseModel.m
```

```

■ ■■■ DataManager.m
■ ■■■ MarketData.m
■ ■■■ Signal.m
■ ■■■ SignalEngine.m
■ ■■■ Backtester.m
■ ■■■ Strategy.m
■ ■■■ Portfolio.m
■ ■■■ yieldCurveSlope.m
■ ■■■ creditSpreadSignal.m
■ ■■■ payrollsMomentum.m
■■■ +Controller/
■ ■■■ BaseController.m
■ ■■■ DataController.m
■ ■■■ StrategyController.m
■ ■■■ UnderweightController.m
■■■ +View/
■ ■■■ BaseView.m
■ ■■■ ChartView.m
■ ■■■ StrategyView.m
■ ■■■ UnderweightApp.m
■■■ +Tests/
■ ■■■ TestMarketData.m
■ ■■■ TestSignal.m
■ ■■■ TestSignalEngine.m
■ ■■■ TestBacktester.m
■ ■■■ TestStrategy.m
■■■ Underweight/ % Equity regime detection module (12 scripts)

```

12.3 Data Caching

PROT uses two complementary caching layers to avoid redundant API calls and support offline use after an initial download:

Cache Layer	Location	TTL	Contents
In-memory Registry	DataManager.Registry (containers.Map)	24h (session-scoped)	FRED series structs keyed by series ID
.mat file cache	cache/mat/*.mat	24h (disk-persistent)	OHLCV timetables per ticker
SQLite metadata	cache/metadata.db	Permanent	Ticker registry: LastDate, Source, MatPath, NumRows

Appendix A: +sigstrat Function Reference

Function	Signature	Returns	Notes
downloadMarketData	mkt = downloadMarketData(D1, D2)	struct: dates, retDates, spxRet, cashRet, spxPrice	Yahoo Finance / Stoog; 24h cache
downloadMacroData	macro = downloadMacroData(targetDates)	timetable: Date + 26 columns (12 raw + 14 derived)	FRED; requires FRED_API_KEY for non-NaN data
buildSignal	[sig, meta] = buildSignal(type, params, mkt, macro)	T×1 double in [-1,+1]; metadata struct	Types: Momentum, EWMA, Ensemble, MacroZ, Mean Reversion, Composite
generateDemoSignals	[data, names] = generateDemoSignals(macro, mkt)	T×5 double; 5-element cell array of names	5 pre-built signals: YieldSlope_Z, VIX_Z, Credit Spread_Z, SPX_Mom_6m, MacroComposite
generateExtendedSignals	[data, names] = generateExtendedSignals(macro, mkt)	T×12 double; 12-element cell array	12 signals: fast/medium/slow speed categories
normalizeSignals	norm = normalizeSignals(raw, method, opts)	T×N double in [-1,+1]	Methods: RobustZ, StandardZ, RollingZ, Percentile, MinMax
mapSignalToWeight	eqWt = mapSignalToWeight(signal, method, params)	T×1 double in [0,1]	Methods: Step, Linear, Sigmoid, PiecewiseLinear, Spline, Power

Function	Signature	Returns	Notes
backtestEquityCash	bt = backtestEquityCash(eqWts, mkt, freq, tc, constraints)	struct: portRet, cumRet, eqWeight, bench6040, benchEq, turnover, dates	Default: freq=21d, tc=10bp; look-ahead safe
computeStrategyPerf	perf = computeStrategyPerf(bt, annFactor)	struct: annReturn, annVol, sharpe, sortino, calmar, maxDD, hitRate, skewness, kurtosis, table, rollingVol, rollingSharpe, monthlyRet, monthlyMatrix	Also computes metrics for 60/40 and 100% equity benchmarks
stressTestStrategy	stress = stressTestStrategy(bt)	table: Episode, dates, Strategy/Bench/Equity returns, AvgEqWeight, MaxDD	6 historical episodes
optimizeComposite	result = optimizeComposite(signals, names, mkt, opts)	struct: weights, thresholds, eqLevels, composite, inPerf, outPerf	70/30 in/out-of-sample split; fmincon with random restarts
riskBudgetStrategy	result = riskBudgetStrategy(signals, names, mkt, macro, opts)	struct: eqWts, allocations, budgets, translations	Optional macro conditioning on any macro series
decisionTreeStrategy	result = decisionTreeStrategy(signals, names, mkt, macro, opts)	struct: eqWts, rulesText, importance, cvMetrics	Modes: RuleBased (user rules) or AutoOptimize (CART + CV)
robustOptimize	result = robustOptimize(signals, names, mkt, opts)	struct: weights, composites, eqWts, cvMetrics, weightStability	K-fold CV + L2 regularisation + turnover penalty; multiple mapping methods

Appendix B: Data Source Reference

B.1 Yahoo Finance (Unofficial JSON API)

```
% URL pattern used by DataManager.fetch_yahoo and sigstrat.downloadMarketData
% POSIX timestamps d1u, d2u from: posixtime(datetime(D, 'InputFormat', 'yyyyMMdd', '
TimeZone', 'UTC'))
url = sprintf(
    'https://query1.finance.yahoo.com/v8/finance/chart/%s?interval=1d&period1;=%s&p
eriod2;=%s&events;=history',
    yfSym, d1u, d2u);
% Ticker mapping: ^ prefix → %5E encoding (e.g., ^GSPC → %5EGSPC)
% Rate limit: ~2 requests/second; 429 responses trigger automatic retry with back
off
```

B.2 FRED REST API

```
% URL pattern for FRED series observations
url = sprintf(
    'https://api.stlouisfed.org/fred/series/observations?series_id=%s&observation;_
start=%s&observation;_end=%s&api;_key=%s&file;_type=json',
    seriesId, startDate, endDate, apiKey);
% Date format: YYYY-MM-DD
% Response: JSON with 'observations' array of {date, value} objects
% Free key: fred.stlouisfed.org/docs/api/api_key.html (rate limit: 120 req/min)
```

B.3 Stooq CSV

```
% URL pattern used by DataManager.fetch_stooq
% Ticker mapping: ^SPX → %5ESPX (URL encoding of caret)
url = sprintf('https://stooq.com/q/d/l/?s=%s&d1;=%s&d2;=%s&i;=d',
    lower(ticker), D1, D2); % D1, D2 in yyyyMMdd format
% Returns: CSV with columns Date,Open,High,Low,Close,Volume
% No authentication required; rate limit: generous (no known hard limit)
```

B.4 Alpha Vantage REST API

```
% URL for daily adjusted OHLCV
url = sprintf(
    'https://www.alphavantage.co/query?function=TIME_SERIES_DAILY_ADJUSTED&symbol;=
%s&outputsize;=full&apikey;=%s',
    ticker, apiKey);
% Free key: www.alphavantage.co/support/#api-key (rate limit: 5 req/min, 500 req
/day)
% Returns: JSON 'Time Series (Daily)' with adjusted close
```

Appendix C: Mathematical Notation

Symbol	Definition
w	$N \times 1$ portfolio weight vector; $\sum w_i = 1$
w_{SAA}	Strategic Asset Allocation weight vector
w_{TAA}	Tactical Asset Allocation (SAA + active overlay) weight vector
w_{diff}	Active weight vector: $w_{TAA} - w_{SAA}$
Σ	$N \times N$ asset return covariance matrix (annualised)
R	$T \times N$ matrix of asset daily returns (rows = time, cols = assets)
r_p	$T \times 1$ portfolio daily return vector: $R \cdot w$
r_f	Risk-free rate per period (annualised convention)
μ	$N \times 1$ expected return vector (annualised)
σ_p	Portfolio annualised volatility: $\sqrt{w' \Sigma w}$
TE	Tracking Error: $\sqrt{(w_{diff}' \Sigma w_{diff})} \cdot \sqrt{252}$
RC_i	Risk contribution of asset i : $w_i \cdot (\Sigma w)_i$
VaR_α	Value at Risk at confidence level α (positive number)
$CVaR_\alpha$	Conditional VaR (Expected Shortfall): $E[-r \mid r < -VaR_\alpha]$
λ	EWMA decay factor (default 0.94 per RiskMetrics standard)
T	Number of time observations in the return series
N	Number of assets in the portfolio
k	Log-moneyness in options: $\ln(K/F)$ where K =strike, F =forward price
$w(k)$	SVI total implied variance as function of log-moneyness